Predicting Home-Based vs. Facility-Based Discharge

A MIMIC-IV self-learning tutorial using early hospitalization data and machine learning

Tutorial goal

- Create a reproducible workflow from raw MIMIC-IV tables to one modeling dataset.
- Compare Logistic Regression, Random Forest, and XGBoost with full vs PCA-reduced lab features.
- Show why threshold selection matters for imbalanced healthcare prediction.

350,699	163,555	21.3%
hospitalizations	patients	facility-based rate

Google Colab Notebook: Self Learning Tutorial.ipynb

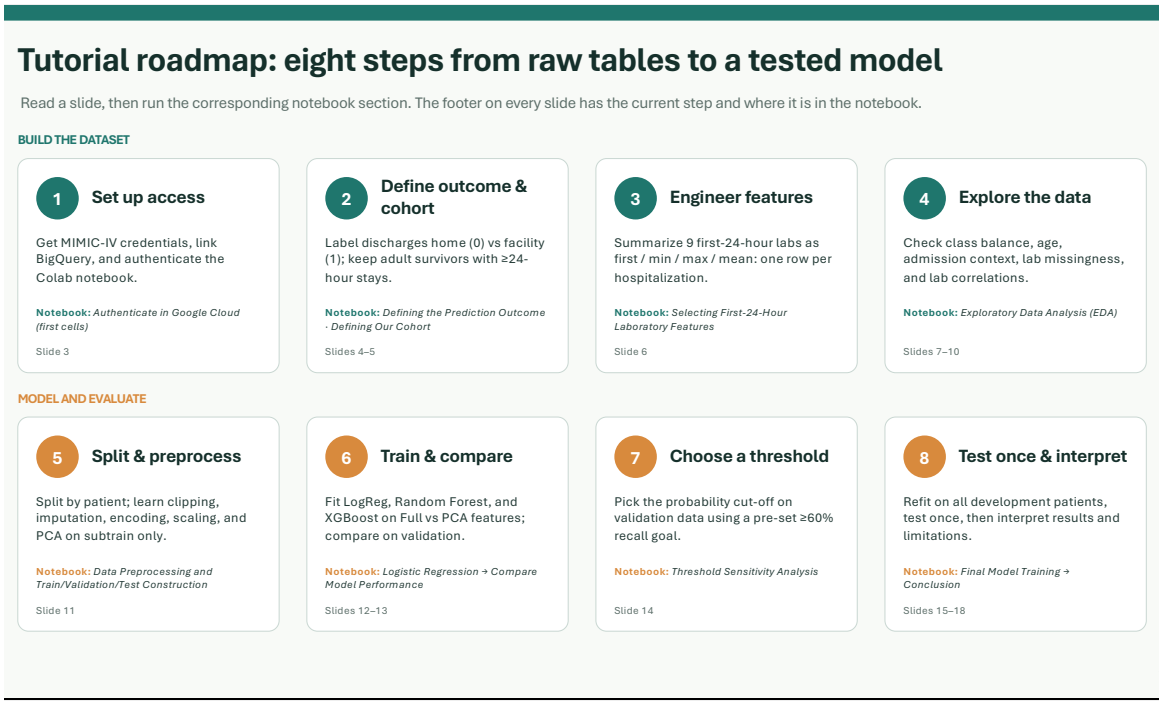
Note: Speaker notes are included in this presentation and can be viewed by downloading as a .pptx file. A separate PDF with slides and notes has also been included.

Notebook link (Google Colab): https://colab.research.google.com/drive/1TArurHyWFT7YFn5s8YWmLn5PCDRu8xFO?usp=sharing
To run or edit it, open the link and choose File > Save a copy in Drive.

This tutorial walks through an end-to-end machine-learning workflow on MIMIC-IV. The question: using only information available 24 hours after admission, can we predict whether an adult patient will be discharged home or to a facility (skilled nursing, rehab, long-term acute care, or assisted living)? Along the way we will practice making decisions that lead to a trustworthy healthcare model: defining a meaningful label, preventing data leakage, splitting by patient, comparing models with metrics suited to an imbalanced outcome, and choosing an operating threshold before touching the test set.

Note that this is a learning project, not a deployable clinical model. The three numbers preview the scale: 350,699 hospitalizations from 163,555 patients, with 21.3% ending in a facility-based discharge.

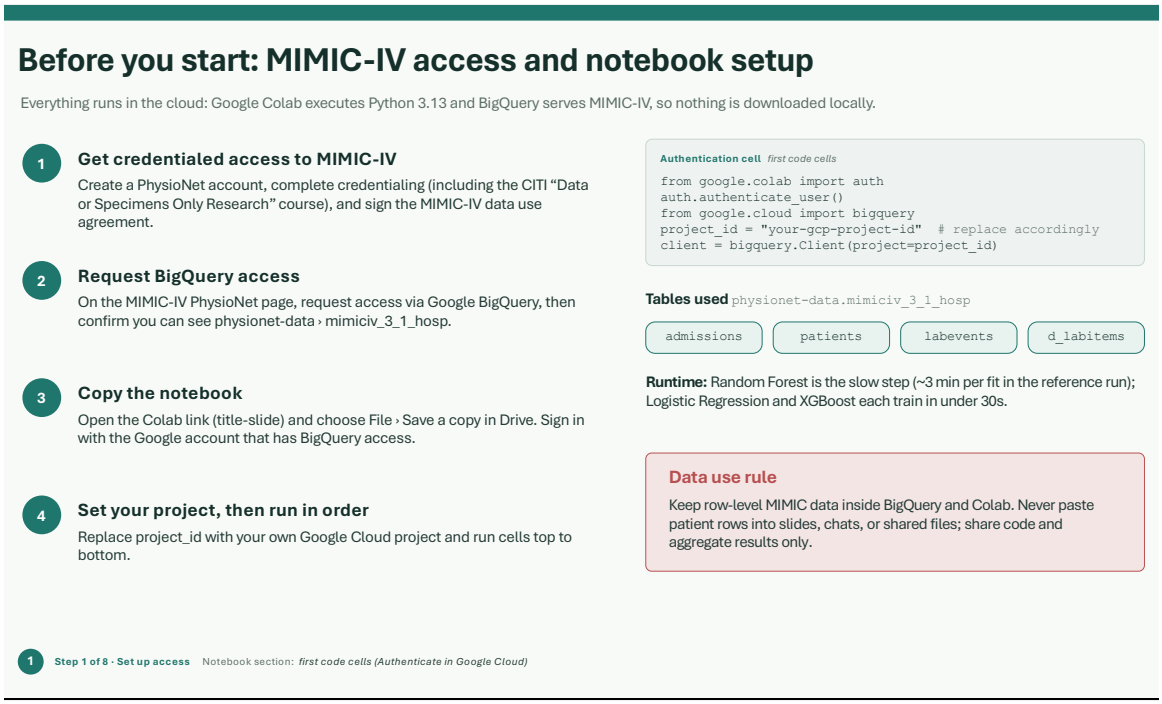
How to use this deck: slide 2 is the roadmap. Every content slide has a footer showing the current step and the matching notebook section (Colab's Table of contents panel lists the same headings). Code on the slides is abridged; the notebook has the full, commented version.



Here is the tutorial workflow in eight steps. Start by reading a slide, and then run the matching notebook section before moving on. The section names on each card are the notebook's own headings.

Steps 1 to 3 build the dataset: get access, define who is in the cohort and what we are predicting, then turn raw lab events into one row per hospitalization. Step 4 is exploratory analysis, which directly shapes the modeling choices (clipping, imputation, and whether to try PCA). Steps 5 to 7 are where most of the methodological care lives: a patient-level split, preprocessing learned only from training data, a fair six-way model comparison, and a threshold chosen on validation data. Step 8 is the single look at the test set, followed by interpretation and limitations.

The workflow order matters as nothing in Steps 5 to 7 uses test-set labels.

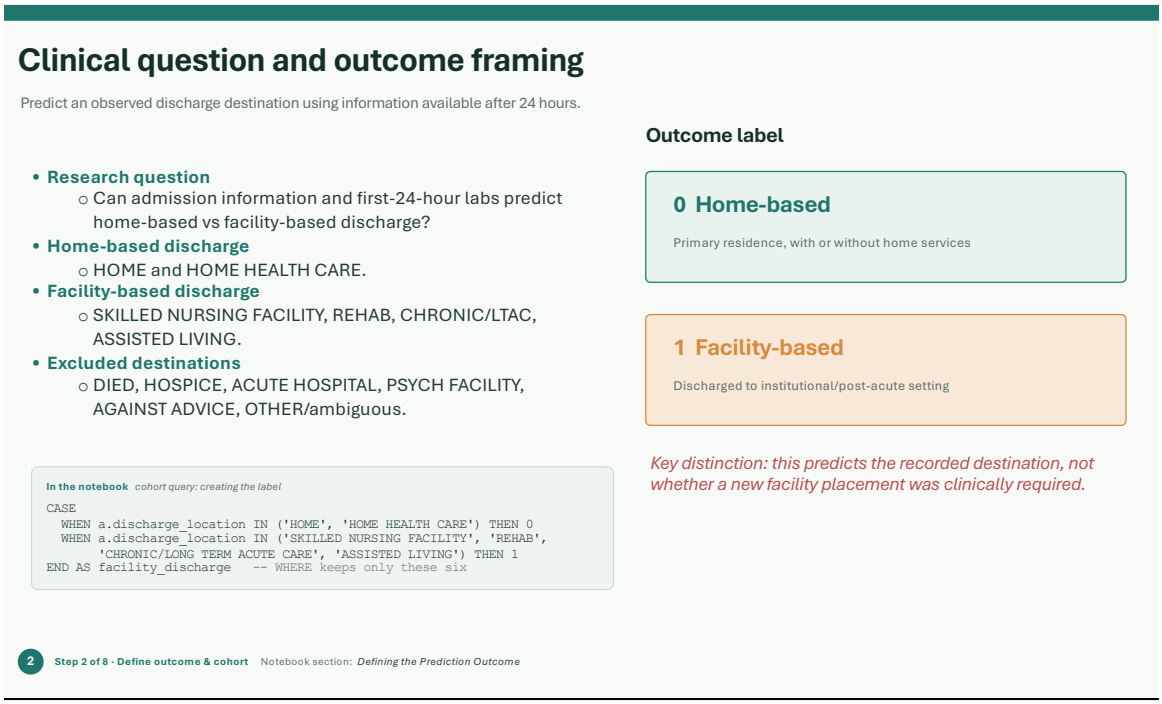


Step 1 is access. MIMIC-IV is credentialed data, so it cannot simply be downloaded. A credentialed PhysioNet account is needed, which includes completing the CITI "Data or Specimens Only Research" course and signing the MIMIC-IV data use agreement. Approval may take up to 7 business days.

Next, on the MIMIC-IV page on PhysioNet, request access through Google BigQuery. The notebook queries the tables directly from the physionet-data project (dataset mimiciv_3_1_hosp), so nothing is downloaded locally. The MIMIC documentation link at the top of the notebook explains how to connect your cloud account. Note that a personal email account may work best as the UT Austin domain has restrictions towards creating a project on Google Cloud Platform.

Then open the Colab link from the title-slide notes, save a copy to your Drive, and sign in with the same Google account that has BigQuery access. In the "Authenticate in Google Cloud" cell, replace project_id with your own Google Cloud project ID; your queries run in that project. Run the cells from top to bottom.

Data use reminder: keep row-level MIMIC data inside Colab and BigQuery. Do not paste patient rows into slides, chats, or shared files; share code and aggregate results only.



Step 2 starts with the label. The notebook first counts every discharge_location value in the admissions table (cell "# Inspect discharge categories") so we can see what exists before grouping anything.

We create a binary target called facility_discharge. HOME and HOME HEALTH CARE become 0, because the patient returns to their own residence, with or without home services. SKILLED NURSING FACILITY, REHAB, CHRONIC/LONG TERM ACUTE CARE, and ASSISTED LIVING become 1. The SQL CASE expression on the slide is how the notebook creates the label inside the cohort query, and the query's WHERE clause keeps only these six destinations. Facility-based discharge was defined as the positive outcome because our goal is to identify patients who may require greater post-discharge care coordination.

Other destinations are excluded rather than forced into a group: death and hospice are end-of-life pathways, acute-hospital transfer means acute care continues, psychiatric facility is a different kind of transition, and categories such as OTHER FACILITY are too ambiguous to label reliably. The cell "# Check ambiguous destination counts" shows how many admissions that affects.

A key distinction: the label records where the patient went, not whether they newly needed facility care. A patient who lived in a nursing home before admission may simply return there. This becomes relevant as admission location turns out to be a strong predictor.

The Cohort: large, moderately imbalanced, and patient-linked

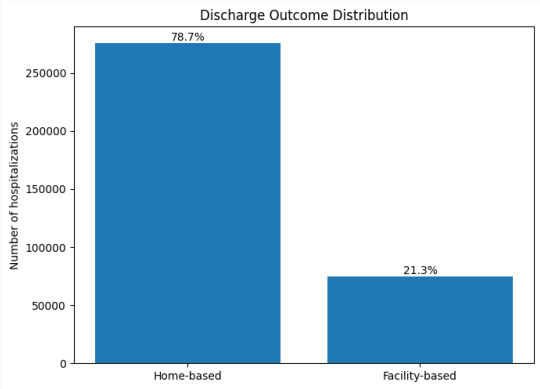


Figure: home-based vs facility-based distribution in eligible admissions.

276,073 home-based admissions

74,626 facility-based admissions

21.3% positive-class prevalence

- Each observation = one hospitalization (hadm_id).
- Adult survivors only, with stays lasting at least 24 hours.
- Outcome groups defined before modeling to avoid ambiguity.
- Class imbalance means accuracy alone can be misleading.

Destination composition

Facility category	Admissions
SNF	52,182
Rehab	13,802
Chronic/LTAC	8,064
Assisted living	578

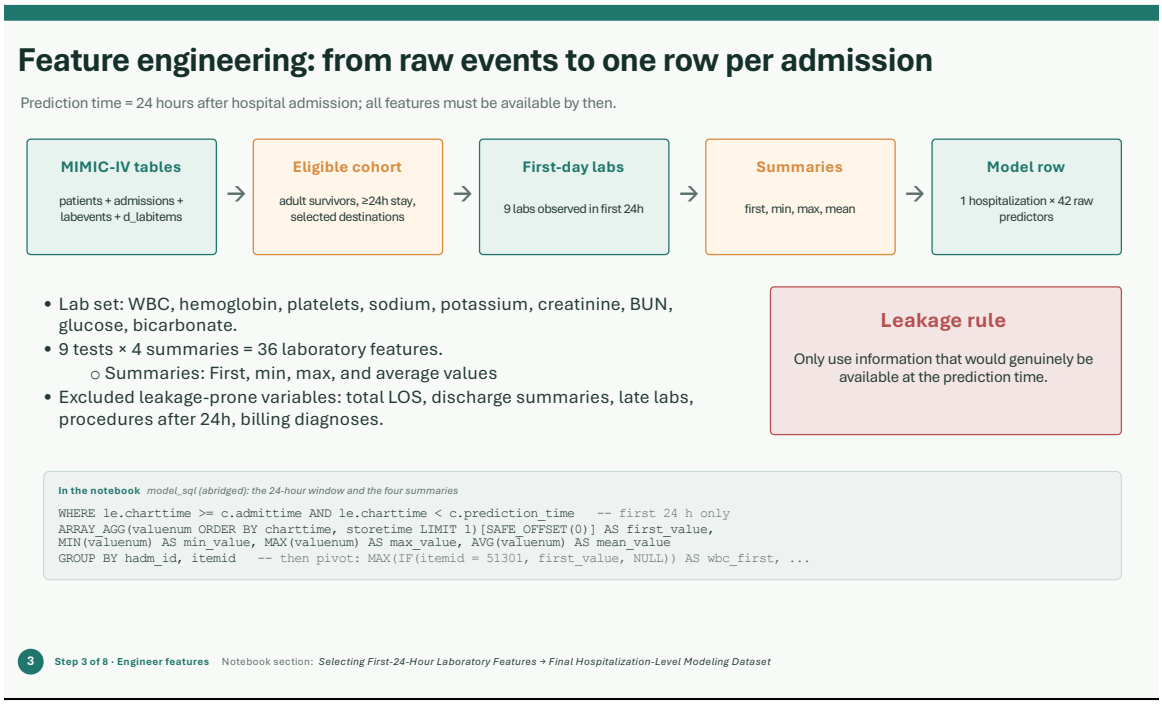
2 **Step 2 of 8 · Define outcome & cohort** Notebook section: *Defining Our Cohort → Cohort Summary*

Continuing Step 2, the cohort query (cell "# Define the cohort once so it can be reused in later BigQuery queries") joins admissions to patients and keeps a hospitalization only if the patient was an adult at admission, survived the stay (hospital_expire_flag = 0), stayed at least 24 hours, and had one of the six labeled destinations. It also computes prediction_time = admittime + 24 hours, which becomes the cutoff for every feature.

Age comes from anchor_age and anchor_year, which is how MIMIC-IV de-identifies dates; computed ages above 89 are grouped at 91. The length-of-stay column is used only to check the 24-hour rule and is not used as a predictor, because total length of stay is only known at discharge.

Check the notebook output against the slide: 350,699 hospitalizations from 163,555 unique patients, 78.7% home-based and 21.3% facility-based. We have the following two takeaways: Since one patient can have several admissions, we must later split by patient, not by row. And because the positive class is about one in five, accuracy alone will mislead: a model that always predicts "home" is already about 79% accurate.

Figure description: Bar chart of home-based (78.7%) versus facility-based (21.3%) admissions, with a table splitting the facility group into skilled nursing (52,182), rehab (13,802), chronic/long-term acute care (8,064), and assisted living (578).



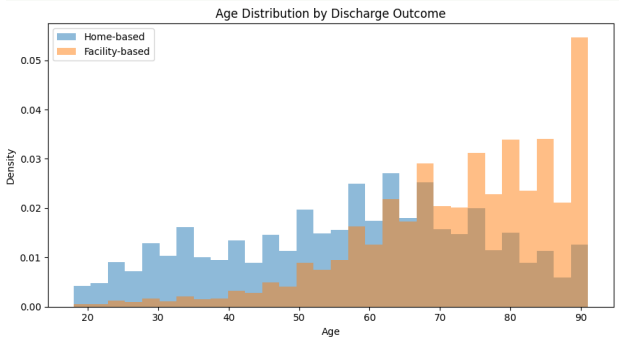
Step 3 turns millions of raw lab events into one row per hospitalization. First we choose nine common blood tests, look up candidate itemids in d_labitems, and confirm them against MIMIC-IV v3.1 (cells "# Identify candidate itemid's from d_labitems" and "# Validate IDs against v3.1 database"). Checking itemids matters because the same test name can appear under several IDs.

The feature query keeps only lab values charted from admittime up to prediction_time (the WHERE clause serves as a leakage guard). For each admission and lab it computes four summaries: the first value (earliest charttime), the minimum, the maximum, and the mean. Nine labs times four summaries gives 36 lab features, pivoted into columns such as wbc_first and wbc_min. Adding age and five admission variables (sex, admission type, admission location, insurance, marital status) gives 42 raw predictors.

Check the result with the cell "# Inspect model_df properties": 350,699 rows and no duplicate hadm_id values. Missing labs are left missing on purpose; they are handled later using training data only.

The leakage rule on the right drives every choice here: if a clinician would not know it 24 hours after admission, the model cannot use it. That is why total length of stay, discharge summaries, later labs and procedures, and billing diagnoses (coded at discharge) are excluded.

EDA: age is strongly associated with discharge destination



igure: age distribution by discharge outcome.

57.5
mean age: home-based

72.9
mean age: facility-based

75
median facility age

- Facility-based admissions are substantially older on average
- Oldest ages are capped/grouped at 91 due to MIMIC de-identification.
- This is an association, not a claim that age alone determines discharge destination.

Group	Mean	Median
Home-based	57.5	59
Facility-based	72.9	75

4 **Step 4 of 8 · Explore the data** Notebook section: *Exploratory Data Analysis (EDA) → Outcome Balance and Age*

Step 4 is exploratory data analysis. The aim is not to prove anything but to understand the data well enough to make good modeling choices.

Age shows the clearest separation: facility-based discharges average 72.9 years (median 75) versus 57.5 years (median 59) for home-based discharges. The histogram uses density rather than counts so the two groups are comparable despite very different sizes. The spike at the right edge is a de-identification artifact: everyone over 89 is recorded as 91, so read it as "90 and older".

Interpretation: age will likely be a strong predictor, but it stands in for things we do not measure directly, such as frailty, comorbidity, mobility, and whether someone can help at home.

Figure description: Overlapping age histograms (density) for home-based and facility-based discharges. The facility-based distribution is shifted toward older ages, with a spike at 91 caused by age capping.

EDA: admission context carries strong signal

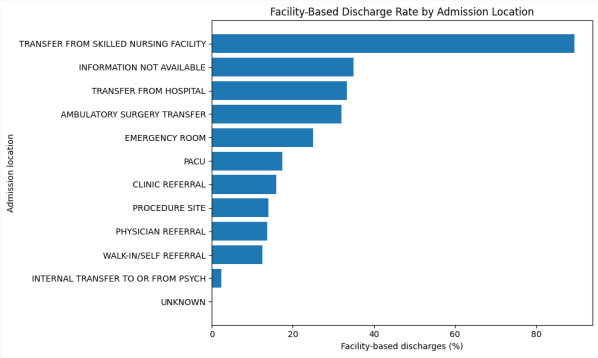


Figure: facility-based discharge rate by admission location; top categories shown.

- Admission location is known at admission and is not considered leakage.
- However, it can act as a proxy for pre-hospital residence or care setting.
- Transfer from SNF: 89.4% facility-based discharge rate.
- Emergency room admissions: 25.1%; physician referral: 13.6%.

Interpretation guardrail

Strong predictive context ≠ causal evidence. A patient admitted from a facility may simply return there.

4 **Step 4 of 8 · Explore the data** Notebook section: *Admission Characteristics and Discharge Destination*

Next, the notebook compares facility-discharge rates across each categorical admission variable. The helper summarize_categorical_outcome counts admissions and the percentage discharged to a facility for each category, showing missing values as UNKNOWN.

Sex shows little difference (about 21% facility-based discharge for both). Admission location matters a lot: 89.4% of patients transferred from a skilled nursing facility go to a facility, compared with 25.1% of emergency-room admissions and 13.6% of physician referrals. Insurance and marital status also differ sharply (for example Medicare 33.5% versus private insurance 8.6%, and widowed 41.9% versus married 16.9%), but both are tangled up with age.

Admission location is legitimate to use because it is known at admission, so it is not leakage. But it partly encodes where the patient lived before, so some of the model's skill will come from that context rather than first-day physiology. A few categories are small (for example fewer than 250 admissions for "information not available" and ambulatory surgery transfers), so treat their rates with caution.

Figure description: Horizontal bar chart of facility-based discharge rate by admission location; transfer from a skilled nursing facility is highest at 89.4%.

EDA: laboratory availability is not random

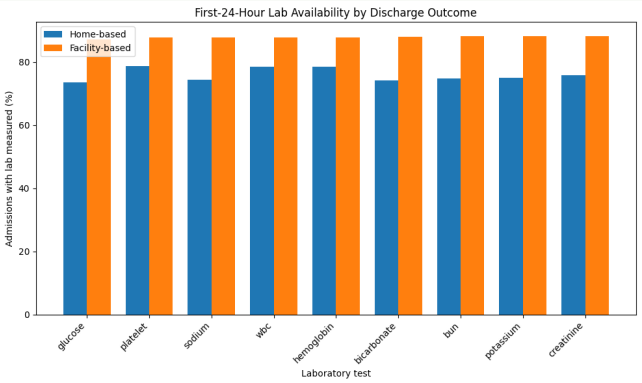


Figure: first-24-hour lab measurement availability by outcome.

- Facility-based admissions had more complete first-day testing across all nine labs.
- Bicarbonate: 74.3% measured in home-based vs 88.0% in facility-based admissions.
- The gap is about 9–14 percentage points on every lab, largest for bicarbonate, glucose, BUN, and sodium.
- Missingness may reflect clinical complexity or testing intensity.

Modeling choice

Primary models use median imputation; adding lab-missingness indicators is a future extension.

4 **Step 4 of 8 · Explore the data** Notebook section: *Laboratory Availability Differs by Discharge Outcome*

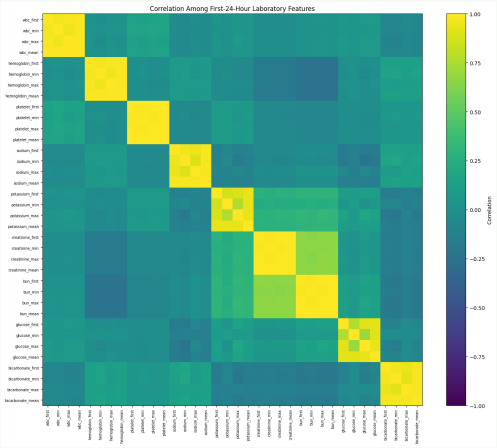
Missing data in EHRs is rarely random. For each of the nine labs, the notebook computes the percentage of admissions with at least one value in the first 24 hours, separately for each outcome group (cell "# Lab availability percentage by discharge outcome").

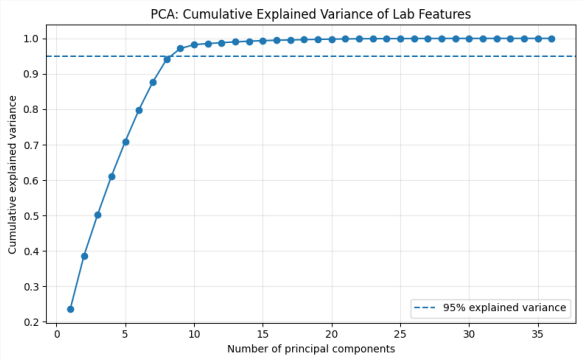
Every lab was measured more often in the facility-based group, by roughly 9 to 14 percentage points. Bicarbonate, for example, was measured in 74.3% of home-based versus 88.0% of facility-based admissions. These are percentage-point gaps, not relative changes. A plausible explanation is that more complex patients get more early testing, so whether a test was ordered is itself a signal.

Our models use median imputation learned from training data, which keeps every admission but hides that "was it ordered?" signal. Adding missingness indicators (SimpleImputer with add_indicator=True) is a good extension to try. The same section also checks extreme values: glucose, for example, has values above 3,000, far beyond its 99th percentile, which motivates the percentile clipping in Step 5.

Figure description: Paired bars for each lab showing the percentage of admissions with a first-24-hour measurement; the facility-based bar is higher for all nine labs.

EDA: correlated lab summaries motivate PCA





Left: 36-feature lab correlation heatmap. Right: PCA cumulative explained variance.

- Many first/min/max/mean summaries for the same lab are highly correlated.
- Top correlations: BUN first vs max = 0.997; platelets min vs mean = 0.996.
- PCA reduces 36 lab variables to 9 components while retaining ≥95% variance.

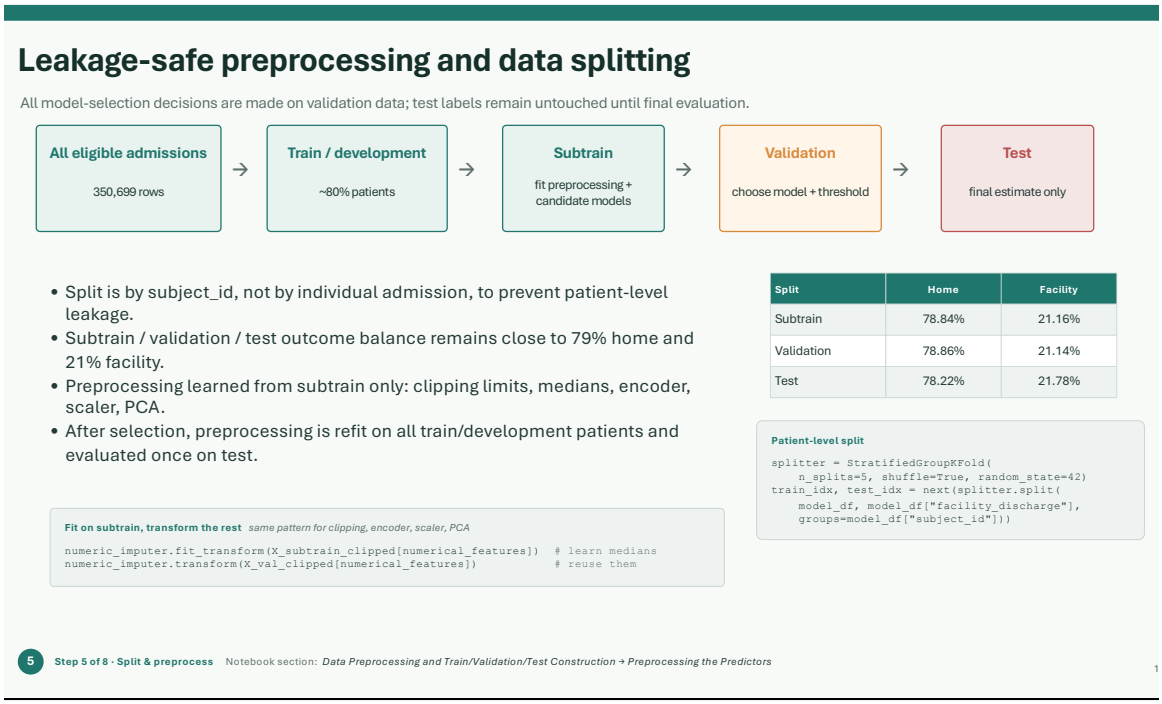
4 **Step 4 of 8 · Explore the data** Notebook section: *Correlated Laboratory Features and Motivation for PCA; PCA fit in Principal Component Analysis*

Since each lab is summarized four ways, many features are near-duplicates. The heatmap shows bright blocks along the diagonal: first, min, max, and mean of the same lab move together. The cell "# Find the strongest absolute pairwise correlations" lists the top pairs, for example BUN first vs max at 0.997.

That redundancy motivates an experiment. Principal Component Analysis (PCA) compresses correlated variables into fewer uncorrelated components. The PCA cells (in the preprocessing section, Step 5) fit PCA only on the standardized subtraining labs and find that 9 components keep at least 95% of the variance of the 36 lab features. We then train every model twice, once with the 36 original lab features and once with the 9 components, and let validation performance decide.

Keep in mind: PCA keeps variance, not necessarily the information that predicts the outcome. A low-variance direction can still matter for prediction.

Figure description: Left, a 36-by-36 correlation heatmap with strong within-lab blocks. Right, cumulative explained variance by number of components, crossing the 95% line at 9 components.



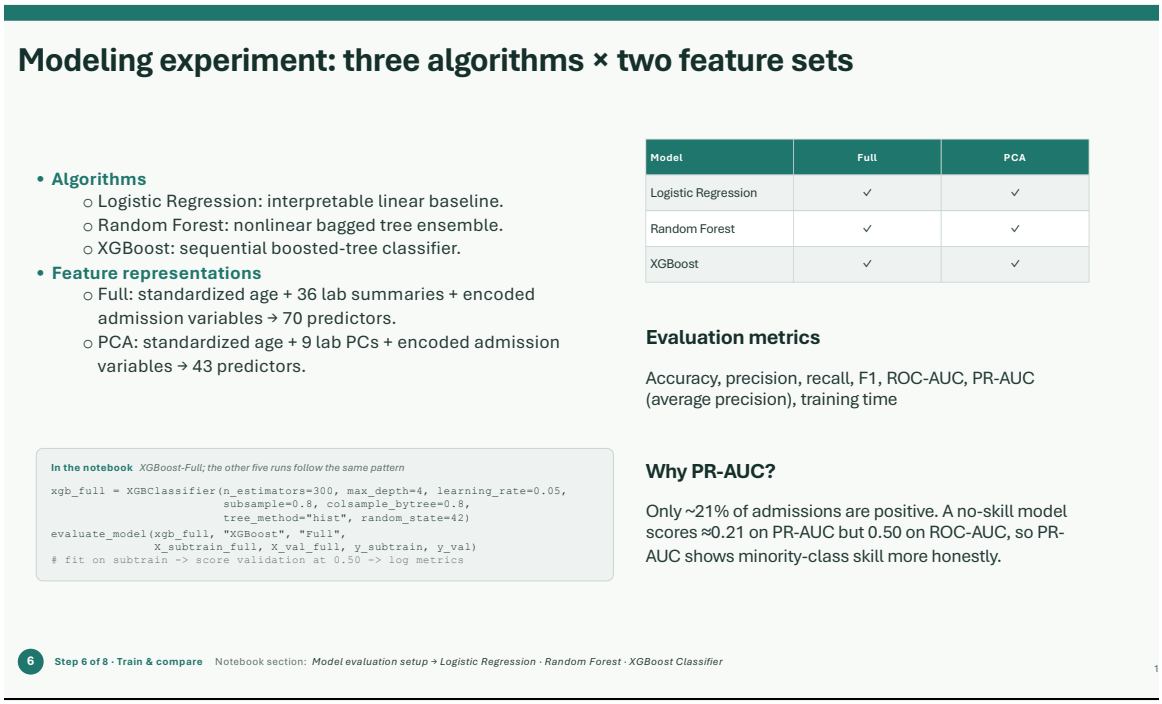
Step 5 contains the key methodology.

First, split by patient. StratifiedGroupKFold with groups = subject_id guarantees that all admissions from one patient land in the same split, while stratification keeps the facility rate near 21% everywhere. One fold becomes the test set (about 20% of patients; 70,123 admissions). The remaining development patients are split again into subtrain and validation. The notebook then asserts that no patient appears in two splits.

We prefer this method over a random row split. If one admission from a patient is in training and another is in test, the model can partly recognize the patient, and test performance looks better than it would on new patients.

Second, every preprocessing step is learned from subtraining data only and then applied unchanged to validation: 1st/99th-percentile clipping of labs, median imputation, "UNKNOWN" for missing categories, one-hot encoding, standardization, and PCA. The code pattern is always the same: fit (or fit_transform) on subtrain, transform everything else. After model and threshold selection are finished, these steps are refit on all development patients and applied once to test.

Figure description: A flow from all admissions to train/development, subtrain, validation, and test; a table confirming about 79% home and 21% facility in each split; and code for the grouped split and the fit/transform pattern.



Step 6 is a controlled experiment: three algorithms times two feature sets, all trained on the same subtraining patients and scored on the same validation patients.

Logistic Regression is the interpretable linear baseline. Random Forest averages many decision trees built on bootstrap samples, which captures nonlinear effects and interactions. XGBoost builds trees sequentially, each correcting the errors of the previous ones, and is often strong on tabular data. The models use fixed, reasonable hyperparameters selected before the validation comparison; no systematic hyperparameter tuning was performed.

To keep the comparison fair, the helper evaluate_model does the same thing for every configuration: fit on subtrain, predict probabilities for validation, apply the default 0.50 threshold, print a classification report and confusion matrix, and record accuracy, precision, recall, F1, ROC-AUC, PR-AUC, and training time.

Why emphasize PR-AUC? ROC-AUC can look good even when a model struggles with a rare class, because it also rewards correctly ranking the many negatives. PR-AUC focuses on the positive class, and its no-skill baseline equals the prevalence (about 0.21 here) rather than 0.5. The notebook computes it with scikit-learn's average_precision_score, the standard summary of a precision-recall curve.

Figure description: Left, the algorithms and feature representations. Right, a grid of the six configurations and the evaluation metrics, with an example model definition and evaluate_model call below.

Validation results: XGBoost-Full is selected

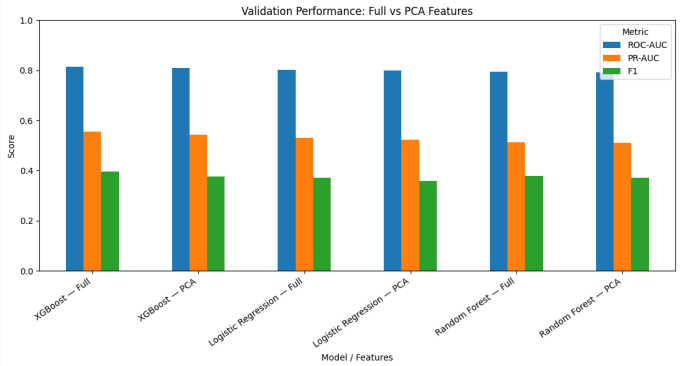


Figure: validation ROC-AUC, PR-AUC, and F1 across all six model configurations.

- XGBoost-Full ranks highest: PR-AUC 0.556 (≈2.6× the 0.21 no-skill baseline), ROC-AUC 0.814.
- Full feature representation slightly outperforms PCA for all three model families.
- PCA preserves variance, but not necessarily all outcome-predictive information.
- Default 0.50 threshold still has low recall: only 28.4% for XGBoost-Full.

Model	Features	PR-AUC	ROC-AUC	Recall @0.50
XGBoost	**Full**	**0.556**	**0.814**	**0.284**
XGBoost	PCA	0.543	0.809	0.266
LogReg	Full	0.531	0.802	0.264
LogReg	PCA	0.524	0.799	0.253
Random Forest	Full	0.515	0.795	0.273
Random Forest	PCA	0.510	0.792	0.270

6 **Step 6 of 8 · Train & compare** Notebook section: *Compare Model Performance → Comparing the Six Model Configurations*

The comparison table (cell "# Compare validation performance across all models") sorts the six runs by PR-AUC, then ROC-AUC. XGBoost with the full feature set wins on both: PR-AUC 0.556 and ROC-AUC 0.814. That PR-AUC is about 2.6 times the no-skill baseline of 0.21, so the model ranks patients meaningfully.

Two patterns are noticeable. First, full lab features beat PCA for all three algorithms, although the gaps are small (0.005 to 0.013 in PR-AUC): PCA made the inputs smaller without making the models better. Second, every model has low recall at the default 0.50 threshold; only 25 to 28% of facility-bound patients are flagged. Since the ranking metrics (ROC-AUC, PR-AUC) are decent, the problem is the cut-off (threshold) rather than the ranking, which is exactly what Step 7 addresses.

Model selection happens here using only validation data. The test set remains untouched.

Figure description: Grouped bars of ROC-AUC, PR-AUC, and F1 for all six configurations. The table lists PR-AUC, ROC-AUC, and recall at the 0.50 threshold for each, with the selected XGBoost-Full row highlighted.

Threshold selection: turning probabilities into workflow decisions

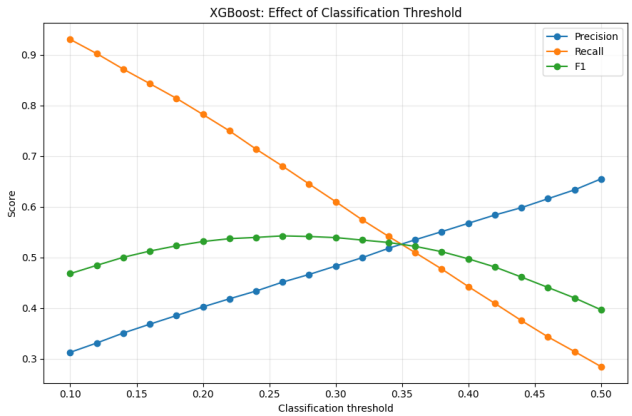


Figure: precision, recall, and F1 for XGBoost-Full across probability thresholds.

- At 0.50, recall is low: 28.4%; only 9.2% of admissions are flagged.
- Lowering the threshold increases recall but reduces precision and specificity.
- Predefined objective: achieve ≥60% recall while keeping threshold as high as possible.
- Validation-selected threshold = 0.30.

Threshold	Prec.	Recall	Specificity	Flagged
0.50	0.654	0.284	0.960	9.2%
0.30	**0.482**	**0.609**	**0.825**	**26.7%**
0.20	0.402	0.781	0.688	41.1%

In the notebook *simplified*

```
# keep thresholds meeting the recall goal; take the highest
eligible = threshold_df[threshold_df["Recall"] >= 0.60]
selected_threshold = eligible["Threshold"].max()   # 0.30
```

7 **Step 7 of 8 · Choose a threshold** Notebook section: *Threshold Sensitivity Analysis → Selecting an Operating Threshold*

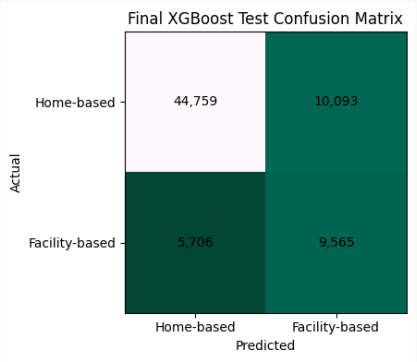
Step 7. A classifier outputs a probability; the threshold turns it into a boolean (yes/no) flag. 0.50 is a convention, not a clinical decision. Here we imagine the model as a screening aid for early discharge planning, where missing a patient who will need a facility is arguably worse than reviewing a few extra patients.

The notebook sweeps thresholds from 0.10 to 0.50 in steps of 0.02 on the validation predictions of XGBoost-Full and records precision, recall, F1, specificity, and "Flagged (%)", the share of all admissions that would be flagged, which approximates review workload. Lowering the threshold raises recall but lowers precision and specificity and increases workload.

We fix the rule before looking at any test results: reach at least 60% recall while keeping the threshold as high as possible. The code keeps thresholds with recall of at least 0.60 and takes the highest, which is 0.30. At 0.30 on validation: precision 0.482, recall 0.609, specificity 0.825, with 26.7% of admissions flagged. A different goal gives a different threshold (for example 0.24 for 70% recall), which is why the rule has to be chosen first. Note that this rule is an illustrative design choice for the tutorial, not a clinically validated requirement.

Figure description: Line plot of precision rising and recall falling as the threshold increases from 0.10 to 0.50, with F1 as a gently curved arc in the middle. The table compares thresholds 0.50, 0.30, and 0.20.

Final held-out test evaluation



Metric	Value
Accuracy	0.775
Balanced accuracy	0.721
Precision	0.487
Recall	0.626
Specificity	0.816
F1	0.548
ROC-AUC	0.814
PR-AUC	0.564
Flagged	28.0%

- Model identifies 62.6% of facility-based discharges.
- Correctly leaves 81.6% of home-based discharges unflagged.
- Precision 48.7%: about half of flagged admissions truly became facility-based.
- Flagged 28.0%: estimated review workload if used as a screening alert.

Figure: final XGBoost test confusion matrix at threshold 0.30.

Confusion matrix counts: TN 44,759 · FP 10,093 · FN 5,706 · TP 9,565

8 Step 8 of 8 · Test once & interpret Notebook section: *Final Model Training → Final Held-Out Test Performance*

Step 8. With the algorithm, feature set, hyperparameters, and threshold all fixed, the notebook refits the preprocessing and the XGBoost model on all development patients (subtrain plus validation), applies them unchanged to the 70,123 test admissions, and evaluates once.

At the 0.30 threshold, test results are close to validation. This is a good sign that the selection did not overfit: recall 0.626, precision 0.487, specificity 0.816, ROC-AUC 0.814, PR-AUC 0.564, with 28.0% of admissions flagged.

Reading the confusion matrix: 9,565 facility-bound patients were correctly flagged (true positives) and 5,706 were missed (false negatives). 44,759 home-bound patients were correctly left unflagged (true negatives), while 10,093 were flagged but went home (false positives). In practical terms, roughly one flag in two is correct, and a team would review about 28 of every 100 admissions.

Figure description: Confusion matrix at threshold 0.30, actual outcome on rows and predicted outcome on columns, with a table of final test metrics.

Why accuracy alone would miss the point

78.2%
accuracy if predicting home for everyone

0%
facility recall for majority baseline

62.6%
facility recall for final model

72.1%
balanced accuracy final model

Threshold	Accuracy	Recall	Precision	Flagged
0.50	0.813	0.291	0.658	9.6%
0.30	0.775	0.626	0.487	28.0%

- Majority baseline looks strong on accuracy but finds no facility-bound patients.
- The selected threshold turns the model into a screening-oriented classifier.
- Use recall, balanced accuracy, F1, PR-AUC, and workload alongside accuracy.

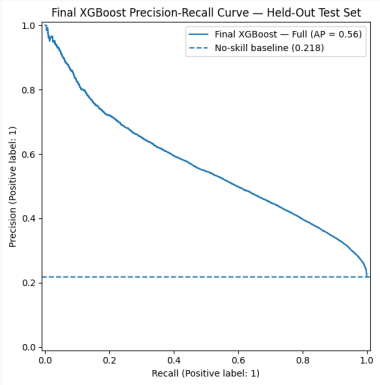


Figure: final test precision-recall curve; no-skill baseline is positive-class prevalence.

8 Step 8 of 8 · Test once & interpret Notebook section: *Effect of the Selected Threshold · Why Accuracy Alone Is Misleading*

This slide answers a basic preliminary question: why is the final accuracy (77.5%) lower than simply predicting "home" for everyone (78.2%)? Because that trivial rule never finds a single facility-bound patient: its recall is 0 and its balanced accuracy is 50%. Accuracy rewards the majority class.

Balanced accuracy averages the recall of each class (here 0.626 and 0.816), so both outcomes count equally; the model reaches 72.1%. The table shows the trade-off on the test set: moving from 0.50 to 0.30 lowers accuracy from 0.813 to 0.775 but more than doubles recall, from 0.291 to 0.626. Ranking metrics (ROC-AUC and PR-AUC) do not change with the threshold; only the operating point moves.

The precision-recall curve sits well above the dashed no-skill line at 0.218, the test-set prevalence. The 0.50-versus-0.30 comparison on the test set is descriptive only; 0.30 was selected from validation results before the test set was evaluated.

Figure description: Four summary numbers comparing the majority baseline with the final model, a table of test metrics at thresholds 0.50 and 0.30, and the test precision-recall curve with a dashed no-skill baseline.

Interpreting the selected XGBoost model



- insurance_Medicare dominates (~34% of total importance); it partly tracks age ≥65, so correlated features can share credit.
- Admission-from-SNF signal supports the idea that pre-admission setting matters.
- Labs such as WBC, BUN, hemoglobin, glucose, and sodium also appear among top predictors.
- Feature importance is predictive, not causal.

Interpretation warning

Associations used by a model do not prove causes of facility discharge.

Figure: top 20 feature importances from final refitted XGBoost model.

8 **Step 8 of 8 · Test once & interpret** Notebook section: *feature-importance cells for the final XGBoost model*

The final model's built-in XGBoost feature importance (gain-based by default, normalized to sum to 1) shows which inputs the trees relied on most. insurance_Medicare stands out at about 34% of total importance, followed by age, admission from a skilled nursing facility, and marital status; several labs (WBC, BUN, hemoglobin, glucose, sodium) also appear.

Medicare eligibility largely tracks age 65 and over, so Medicare and age carry overlapping information, and tree models can split credit between correlated features somewhat arbitrarily. Gain importance also says nothing about direction, that is, whether a feature raises or lowers predicted risk. And none of it is causal: insurance does not send anyone to a nursing home.

For a stronger picture, compute permutation importance on the validation set (scoring="average_precision") or use SHAP values.

Figure description: Horizontal bar chart of the top 20 feature importances from the final refitted XGBoost model; insurance_Medicare is by far the largest bar.

Conclusion: limitations and takeaways

- **Limitations**
  - Outcome is recorded destination, not confirmed new need for facility care.
  - Important factors are missing: functional status, mobility, cognition, social support, therapy evaluations.
  - Admission location may proxy for prior residence; lab missingness is informative but not explicitly modeled.
  - Threshold is illustrative; real deployment would require clinical workflow validation and external validation.
- **Takeaways**
  - Early hospitalization data contain meaningful signal for discharge destination.
  - Full lab features slightly outperform PCA across all models.
  - XGBoost-Full performs best on validation and generalizes consistently on test.
  - Threshold choice changes clinical usefulness more than accuracy alone suggests.

Final message

A healthcare ML model is not just an AUC. It is a pipeline that involves defining a clinically meaningful target, preventing leakage, evaluation on patient-disjoint data, choosing an operating threshold, and interpreting tradeoffs.

8 Step 8 of 8 · Test once & interpret Notebook section: *Limitations · Conclusion*

1

To close, we separate what the tutorial shows from what it does not. The label is the recorded destination, not a new clinical need. The predictors leave out what discharge planners actually weigh (function, mobility, cognition, social support, therapy evaluations), so some of the model's signal arrives through proxies such as age, insurance, and admission location. Lab missingness is informative but not modeled. The 0.30 threshold is illustrative; real use would need clinical workflow input, a check that predicted probabilities are well calibrated, and external validation at another hospital.

The takeaways are mostly about method: define the target carefully, stop features at the prediction time, split by patient, choose the model and threshold on validation data, and look at the test set once. Early data does carry real signal (test ROC-AUC 0.81, PR-AUC 0.56), full lab features slightly beat PCA, and the threshold choice moved recall far more than switching between the top models did.

Natural next steps are adding lab-missingness indicators, comparing class weighting with the threshold moving used here, tuning hyperparameters with patient-grouped cross-validation on development data only, and checking the importances with permutation importance or SHAP.